

13. REST WebデータベースAPI

13.1. Webデータベース：レコードの取得（GET）

レコードをクエリで条件を指定して取得できます。

- 一度に取得できるレコードは 100 件までです。（100 件以上取得する場合はレコードの取得範囲を変えて複数回実行してください）
- ソート順に指定できるフィールドはフィールド設定で「ソート対象」を指定できるフィールドです。
- 検索条件の指定方法、検索条件に指定できるフィールド、論理演算子、比較演算子は絞り込み条件と同じです。

24

☒ すべての条件を満たす
 ☐ いずれかの条件を満たす

項目	条件	
☐ 受付者	大木 明	と同じ
☐ 進捗区分	クレーム対応済	と異なる

条件を追加

選択した行を編集
 選択した行を削除

14

☐ かつ
 ☐ または
 ブロックを削除

項目	条件	
☐ 対策部署	営業部	と同じ
☐ 対策部署		未入力

条件を追加

選択した行を編集
 選択した行を削除

以降、①は「ブロック毎の論理演算子」、②は「フィールド毎の論理演算子」と記述します。

13.1.1. HTTPメソッド

GET

13.1.2. URI

- ### 1. パラメータを HTTP のクエリ文字列で送信する場合

http(s)://(POWER EGGサーバ)/pe4j/api/rest/v1/xdb/q/records.json

2. パラメータを JSON形式で送信する場合

http(s)://(POWER EGGサーバ) /pe4j/api/rest/v1/xdb/records.json

❶ 注釈

GETでのリクエストボディの指定をサポートしていないクライアントでは使用できません

13.1.3. リクエストパラメータ

パラメータ名	指定する値	必須	説明
system	数値	省略可	システム区分を指定します。(0:WebDB,1:汎用申請,2:CRM) ※省略時は0が指定されます。
database	文字列	必須	データベース名を指定します。(親フォルダ名)/(フォルダ名)/(データベース名)
query	JSON形式の配列	省略可	レスポンスに含めるレコードの条件を指定するクエリ文字列を指定します。 ※省略時は全レコードが対象になります。
order	JSON形式の配列	省略可	レコードの並び順を指定します。 ※省略時はNoの昇順でソートされます。
limit	数値	省略可	指定した数だけ、レコードを返却します。 ※省略時は100が指定されます。
offset	数値	省略可	指定した数だけ、出力するレコードをスキップします。 ※省略時は0が指定されます。

13.1.4. 「query」パラメータ

パラメータ名	指定する値	必須	説明
blockConj	文字列	省略可	ブロック毎の論理演算子を指定します。 ("and","or") ※省略時は"and"が指定されます。
itemConj	文字列	省略可	フィールド毎の論理演算子を指定します。 ("and","or") ※省略時は"and"が指定されます。
items	Object	必須	検索条件を指定します・

13.1.5. 「items」パラメータ

パラメータ名	指定する値	必須	説明
field	文字列	必須	検索条件に指定するフィールド名を指定します。

パラメータ名	指定する値	必須	説明																																			
opr	文字列	必須	比較演算子を指定します。																																			
			比較演算子	説明	=	同じ	!=	異なる	^	始まる	~	含む	!~	含まない	>	より大きい	<	より小さい	>=	以上	<=	以下	n	未入力	!n	入力済	~=	配下（部門選択で利用）	c=	企業が同じ（お客様選択で利用）	b=	部門が同じ（お客様選択で利用）	e=	担当者が同じ（お客様選択で利用）	true	チェック済（フラグで利用）	false	未チェック（フラグで利用）
			比較演算子	説明																																		
			=	同じ																																		
			!=	異なる																																		
			^	始まる																																		
			~	含む																																		
			!~	含まない																																		
			>	より大きい																																		
			<	より小さい																																		
			>=	以上																																		
			<=	以下																																		
			n	未入力																																		
			!n	入力済																																		
			~=	配下（部門選択で利用）																																		
			c=	企業が同じ（お客様選択で利用）																																		
			b=	部門が同じ（お客様選択で利用）																																		
			e=	担当者が同じ（お客様選択で利用）																																		
			true	チェック済（フラグで利用）																																		
			false	未チェック（フラグで利用）																																		
code	真偽値	省略可	検索条件にDB参照、DB参照コード入力、WebDB参照、WebDB参照コード入力を使用する場合に指定します。（true:コードで比較,false:名称で比較） ※省略時はfalseが指定されます。																																			

パラメータ名	指定する値	必須	説明	
value	文字列	省略可	検索する文字列を指定します。oprによって、必須/任意が切り替わります	
			フィールド	指定方法
			日付、申請日、決裁日	YYYY-MM-DD形式で指定 例) 2015-02-05
			年月	YYYY-MM形式で指定 例) 2015-02
			時刻	HH:MM形式で指定 例) 10:00
			日付時刻、"日付時刻計算	YYYY-MM-DDTHH:MM±HH:MM形式で指定
			例) 2015-02-05T10:30+09:00"	
			登録日時、"最終更新日時	YYYY-MM-DDTHH:MM:SS±HH:MM形式で指定
			例) 2015-02-05T10:30:15+09:00"	
			会社選択	外部会社コードを指定 例) friend00
			部門選択、申請部門	(外部会社コード):(外部部門コード1)形式で指定 例) friend00:eigyo
			社員選択、複数社員選択	(外部会社コード):(外部社員コード1)形式で指定 例) friend00:ooishi
			商品選択	(外部会社コード):(商品コード)形式で指定 例) friend00:poweregg
			お客様選択、顧客リレーション	(企業ID):(部署ID):(担当者ID)形式で指定 例) 1:2:3
登録者、最終更新者、申請者、決裁者	ユーザIDを指定 例) ooishi			
それ以外	文字列を指定			

13.1.6. 「order」パラメータ

パラメータ名	指定する値	必須	説明
field	文字列	必須	ソート条件するフィールド名を指定します。

パラメータ名	指定する値	必須	説明
desc	真偽値	省略可	並び順を指定します。（true:降順,false:昇順） ※省略時はfalseが指定されます。

13.1.7. リクエストの例

1. レコード取得条件

- 対象のデータベースは「ポイント管理型/クレーム管理」
- 並び順は「No」の「降順」
- レコードの取得範囲は「50」件目から「100」件
- 検索条件は以下のとおり

☒ すべての条件を満たす
☐ いずれかの条件を満たす

項目	条件	
☐ 受付者	大木 明	と同じ
☐ 進捗区分	クレーム対応済	と異なる

条件を追加

選択した行を編集
 選択した行を削除

☒ かつ
☐ または

ブロックを削除

☐ すべての条件を満たす
☒ いずれかの条件を満たす

項目	条件	
☐ 対策部署	営業部	と同じ
☐ 対策部署		未入力

条件を追加

選択した行を編集
 選択した行を削除

2. パラメータを HTTP のクエリ文字列で送信する場合

HTTPのクエリ文字列の例

```
system=0&database=%EF%BE%8C%EF%BE%9F%EF%BE%9B%EF%BD%BE%EF%BD%BD%E7%AE%A1%E7%90%86%E5%9E%8B%2F%
```

<URLエンコーディングする前>

```
system=0&database=7° 0tλ管理型/クレーム管理DB&query=[{"itemConj":"and","items":[{"field":"受付者","opr":"=", "value":"friend00:ooki"}, {"field":"進捗区分","opr":"!=", "value":"クレーム対応済"}]}, {"blockConj":"and","itemConj":"or","items":[{"field":"対応部署","opr":"=", "value":"friend00:eigyo"}, {"field":"対応部署","opr":"n"}]}]&order=[{"field":"No","desc":true}]&limit=100&offset=50
```

リクエストヘッダを含んだ例

```
GET /pe4j/api/rest/v1/xdb/q/records.json?(HTTPのクエリ文字列) HTTP/1.1
Host: pe280demo.poweregg.net
X-API-Key: 7706d1ce-01a0-463b-a162-d389cec21eb1
```

※Content-Typeヘッダは不要です。

3. パラメータをJSON形式で送信する場合

リクエストヘッダの例

```
GET /pe4j/api/rest/v1/xdb/records.json HTTP/1.1
Host: pe280demo.poweregg.net
X-API-Key: 7706d1ce-01a0-463b-a162-d389cec21eb1
Content-Type: application/json
```

※Content-Type に application/json を指定して下さい。
指定しない場合は JSON が解釈できないため、実行時エラーとなります。

リクエストボディの例

```
{
  "system": 0,
  "database": "ﾌﾟﾛｸﾞﾗﾑ管理型/ｸﾚｰﾑ管理DB",
  "query": [
    {
      "itemConj": "and",
      "items": [
        {
          "field": "受付者",
          "opr": "=",
          "value": "friend00:ooki"
        },
        {
          "field": "進捗区分",
          "opr": "!=",
          "value": "ｸﾚｰﾑ対応済"
        }
      ]
    },
    {
      "blockConj": "and",
      "itemConj": "or",
      "items": [
        {
          "field": "対応部署",
          "opr": "=",
          "value": "friend00:eigyo"
        },
        {
          "field": "対応部署",
          "opr": "n"
        }
      ]
    }
  ],
  "order": [
    {
      "field": "No",
      "desc": true
    }
  ],
  "limit": 100,
  "offset": 50
}
```

13.1.8. レスポンスの例

レコードのリストが JSON 配列で返されます。

※各フィールドの形式については後述「フィールド形式」をご確認ください

```
{
  "count": 2,
  "records": [
    {
      "No": {
        "type": 85,
        "value": 2
      },
      "登録日時": {
        "type": 86,
        "value": "2015-01-15T19:21:14+09:00"
      },
      "登録者": {
        "type": 87,
        "value": {
          "code": "ooishi",
          "name": "大石 学"
        }
      },
      "最終更新日時": {
        "type": 88,
        "value": "2015-01-15T19:21:14+09:00"
      },
      "最終更新者": {
        "type": 89,
        "value": {
          "code": "ooishi",
          "name": "大石 学"
        }
      },
      "受付日時": {
        "type": 58,
        "value": "2015-01-10T10:00+09:00"
      },
      "受付者": {
        "type": 52,
        "value": {
          "corpCode": "friend00",
          "deptCode": "system",
          "empCode": "ooishi",
          "name": "大石 学"
        }
      },
      "お客様": {
        "type": 1,
        "value": "日之出食品株式会社 岸様"
      },
      "進捗区分": {
        "type": 22,
        "value": "クレーム対応済"
      },
      "対策部署": {
        "type": 51,
        "value": {
          "corpCode": "friend00",
          "deptCode": "eigyo",
          "name": "営業部"
        }
      },
      "添付ファイル": {
        "type": 55,
        "value": [
          {
```

```
      "fileKey":
"Od/bdrkdy3ZfD4NaYGHGxQJXKeyXzYZRxdP+LU+CFDmpP5FI2bx7Y4Mq008bTQ3p",
      "name": "問い合わせ.xlsx",
      "size": 120000
    }
  ],
},
"details": [
  {
    "対応日時": {
      "type": 58,
      "value": "2015-01-11T14:00+09:00"
    },
    "対応者": {
      "type": 52,
      "value": {
        "corpCode": "friend00",
        "deptCode": "eigy01",
        "empCode": "ooki",
        "name": "大木 明"
      }
    },
    "対応内容": {
      "type": 11,
      "value": "初期対応"
    }
  },
  {
    "対応日時": {
      "type": 58,
      "value": "2015-01-12T16:00+09:00"
    },
    "対応者": {
      "type": 52,
      "value": {
        "corpCode": "friend00",
        "deptCode": "eigy01",
        "empCode": "doi",
        "name": "土井 勤"
      }
    },
    "対応内容": {
      "type": 11,
      "value": "お詫び"
    }
  }
],
},
{
  "No": {
    "type": 85,
    "value": 1
  },
  "登録日時": {
    "type": 86,
    "value": "2015-01-15T19:21:14+09:00"
  },
  "登録者": {
    "type": 87,
    "value": {
      "code": "ooishi",
      "name": "大石 学"
    }
  }
},
}
```

```

    "最終更新日時": {
      "type": 88,
      "value": "2015-01-15T19:21:14+09:00"
    },
    "最終更新者": {
      "type": 89,
      "value": {
        "code": "ooishi",
        "name": "大石 学"
      }
    },
    "受付日時": {
      "type": 58,
      "value": "2015-01-10T10:00+09:00"
    },
    "受付者": {
      "type": 52,
      "value": {
        "corpCode": "friend00",
        "deptCode": "system",
        "empCode": "ooishi",
        "name": "梨田 義男"
      }
    },
    "お客様": {
      "type": 1,
      "value": "株式会社ザッツ製作所様"
    },
    "進捗区分": {
      "type": 22,
      "value": "クレーム対応中"
    },
    "対策部署": {
      "type": 51,
      "value": {
        "corpCode": null,
        "deptCode": null,
        "name": null
      }
    },
    "添付ファイル": {
      "type": 55,
      "value": []
    },
    "details": []
  }
}

```

13.2. Webデータベース：レコードの登録（POST）

1件のレコードを登録できます。

13.2.1. HTTPメソッド

POST

13.2.2. URI

http(s)://(POWER EGGサーバ)/pe4j/api/rest/v1/xdb/record.json

13.2.3. リクエストパラメータ

パラメータ名	指定する値	必須	説明
system	数値	省略可	システム区分を指定します。 (0:WebDB,2:CRM) ※省略時は0が指定されます。
database	文字列	必須	データベース名を指定します。(親フォルダ名)/(フォルダ名)/(データベース名)
notification	真偽値	省略可	通知の有無を指定します。(true:通知する,false:通知しない) ※省略時はfalseが指定されます。
notifyfollow	真偽値	省略可	フォロー通知の有無を指定します。(true:通知する,false:通知しない) ※省略時はfalseが指定されます。
external	真偽値	省略可	外部連携（アドオン呼び出し）の有無を指定します。(true:実行する,false:実行しない) ※省略時はtrueが指定されます。
record	Object	必須	レコードの情報（フィールド名とフィールドの値）をオブジェクトで指定します。 ※フィールド値の仕様についてはフィールドの形式により異なります。 詳細については、後述「フィールド形式」をご確認ください。 ※明細行（details）は省略可能です。

リクエストボディの構造

```
{
  "system": (システム区分),
  "database": "(データベース名)",
  "notification": (通知の有無),
  "notifyfollow": (フォロー通知の有無),
  "external": (外部連携の有無),
  "record": {
    "(フィールド名)": {
      "value": "(フィールド値)"
    },
    "details": [
      {
        "(明細行フィールド名)": {
          "value": "(フィールド値)"
        }
      }
    ]
  }
}
```

13.2.4. リクエストの例

1. レコード登録条件

- 対象のデータベースは「プロセス管理型/クレーム管理」
- 通知の有無は「通知する」
- フォロー通知の有無は「通知しない」
- 外部連携の有無は「実行しない」
- 登録するフィールドは以下のとおり

レコード

フィールド名	フィールド型	値
登録日時	(システム標準項目)	2015-01-15T19:21:14+09:00
登録者	(システム標準項目)	大石 学(ooishi)
最終更新日時	(システム標準項目)	2015-01-15T19:21:14+09:00
最終更新者	(システム標準項目)	大石 学(ooishi)
受付日時	日付時刻選択	2015-01-10T10:00+09:00
受付者	社員選択	フレンド商事(friend00) システム室(system)
梨田 義男(nakanishi)		
進捗区分	ラジオボタン	クレーム対応済
お客様	文字列（1行）	日之出食品株式会社 岸様
対策部署	部署選択	フレンド商事(friend00) 営業部(eigyō)
添付ファイル	添付ファイル	問い合わせ.xlsx(74b2a870-6bd4-413c-a467-c4c83f6da1f0) ※添付ファイルを登録する場合は事前に「ファイルアップロードAPI」にて、fileKeyを取得する必要があります。

<明細行:1件目>

フィールド名	フィールド型	値
対応日時	日付時刻選択	2015-01-11T14:00+09:00
対応者	社員選択	フレンド商事(friend00) 第一営業課(eigyō1) 大木 明(ooki)
対応内容	文字列（複数行）	初期対応

<明細行:2件目>

フィールド名	フィールド型	値
対応日時	日付時刻選択	2015-01-12T16:00+09:00
対応者	社員選択	フレンド商事(friend00) 第一営業課(eigyō1) 土井 勤(doi)
対応内容	文字列（複数行）	お詫び

2. リクエストヘッダの例

```
POST /pe4j/api/rest/v1/xdb/record.json HTTP/1.1
Host: pe280demo.poweregg.net
X-API-Key: 7706d1ce-01a0-463b-a162-d389cec21eb1
Content-Type: application/json
```

❗ 注釈

Content-Type に application/json を指定して下さい。指定しない場合は JSON が解釈できないため、実行時エラーとなります。

3. リクエストボディの例

```
{
  "system": 0,
  "database": "フンド管理型/クレーム管理DB",
  "notification": true,
  "notifyfollow": false,
  "external": false,
  "record": {
    "登録日時": {
      "value": "2015-01-15T19:21:14+09:00"
    },
    "登録者": {
      "value": {
        "code": "ooishi"
      }
    },
    "最終更新日時": {
      "value": "2015-01-15T19:21:14+09:00"
    },
    "最終更新者": {
      "value": {
        "code": "ooishi"
      }
    },
    "受付日時": {
      "value": "2015-01-10T10:00+09:00"
    },
    "受付者": {
      "value": {
        "corpCode": "friend00",
        "deptCode": "system",
        "empCode": "nakanishi"
      }
    },
    "お客様": {
      "value": "日之出食品株式会社 岸様"
    },
    "進捗区分": {
      "value": "クレーム対応済"
    },
    "対策部署": {
      "value": {
        "corpCode": "friend00",
        "deptCode": "eigyō"
      }
    },
    "添付ファイル": {
      "value": [
        {
          "fileKey": "74b2a870-6bd4-413c-a467-c4c83f6da1f0"
        }
      ]
    },
    "details": [
      {
        "対応日時": {
          "value": "2015-01-11T14:00+09:00"
        },
        "対応者": {
          "value": {
            "corpCode": "friend00",
            "deptCode": "eigyō1",
            "empCode": "ooki"
          }
        }
      }
    ]
  }
}
```

```

    },
    "対応内容": {
      "value": "初期対応"
    }
  },
  {
    "対応日時": {
      "value": "2015-01-12T16:00+09:00"
    },
    "対応者": {
      "value": {
        "corpCode": "friend00",
        "deptCode": "eigy01",
        "empCode": "doi"
      }
    },
    "対応内容": {
      "value": "お詫び"
    }
  }
]
}

```

13.2.5. レスポンスの例

登録したレコードのNoが返されます。

```

{
  "No": 1
}

```

※外部連携を実行する場合に「レコードの登録、変更、削除時」の処理で警告が返された場合は警告メッセージが設定されます。:

```

{
  "No": 1,
  "messages": [
    "警告メッセージ"
  ]
}

```

13.3. Webデータベース：レコードの更新（PUT）

1件のレコードを更新できます。

13.3.1. HTTPメソッド

PUT

13.3.2. URI

http(s)://(POWER EGGサーバ)/pe4j/api/rest/v1/xdb/record.json

13.3.3. リクエストパラメータ

パラメータ名	指定する値	必須	説明
system	数値	省略可	システム区分を指定します。 (0:WebDB,2:CRM) ※省略時は0が指定されます。
database	文字列	必須	データベース名を指定します。(親フォルダ名)/(フォルダ名)/(データベース名)
notification	真偽値	省略可	通知の有無を指定します。(true:通知する,false:通知しない) ※省略時はfalseが指定されます。
notifyfollow	真偽値	省略可	フォロー通知の有無を指定します。(true:通知する,false:通知しない) ※省略時はfalseが指定されます。
external	真偽値	省略可	外部連携（アドオン呼び出し）の有無を指定します。(true:実行する,false:実行しない) ※省略時はtrueが指定されます。
No	数値	必須	レコードNoを指定します。
record	Object	必須	レコードの情報（フィールド名とフィールドの値）をオブジェクトで指定します。 ※フィールド値の仕様についてはフィールドの形式により異なります。 詳細については、後述「フィールド形式」をご確認ください。 ※明細行（details）は省略可能です。 ※明細行は全置換されます。（差分で更新できません）

リクエストボディの構造

```
{
  "system": (システム区分),
  "database": "(データベース名)",
  "notification": (通知の有無),
  "notifyfollow": (フォロー通知の有無),
  "external": (外部連携の有無),
  "No": (レコードNo),
  "record": {
    "(フィールド名)": {
      "value": "(フィールド値)"
    },
    "details": [
      {
        "(明細行フィールド名)": {
          "value": "(フィールド値)"
        }
      }
    ]
  }
}
```

13.3.4. リクエストの例

1. レコード更新条件

- 対象のデータベースは「ポイント管理型/クレーム管理」
- 通知の有無は「通知する」
- フォロー通知の有無は「通知しない」
- 外部連携の有無は「実行しない」
- 更新するレコードNoは「1」
- 更新するフィールドは以下のとおり

フィールド名	フィールド型	値
最終更新日時	(システム標準項目)	2015-01-20T10:05:14+09:00
最終更新者	(システム標準項目)	中西 義男(nakanishi)
進捗区分	ラジオボタン	対策検討依頼済

2. リクエストヘッダの例

```
PUT /pe4j/api/rest/v1/xdb/record.json HTTP/1.1
Host: pe280demo.poweregg.net
X-API-Key: 7706d1ce-01a0-463b-a162-d389cec21eb1
Content-Type: application/json
```

❶ 注釈

Content-Type に application/json を指定して下さい。指定しない場合は JSON が解釈できないため、実行時エラーとなります。

3. リクエストボディの例

```
{
  "system": 0,
  "database": "フロンティア管理型/クレーム管理DB",
  "notification": true,
  "notifyfollow": false,
  "external": false,
  "No": 1,
  "record": {
    "最終更新日時": {
      "value": "2015-01-20T10:05:14+09:00"
    },
    "最終更新者": {
      "value": {
        "code": "nakanishi"
      }
    },
    "進捗区分": {
      "value": "対策検討依頼済"
    }
  }
}
```

13.3.5. レスポンスの例

処理成功時は空の JSON データが返されます。

```
{}
```

❗ 注釈

外部連携を実行する場合に「レコードの登録、変更、削除時」の処理で警告が返された場合は警告メッセージが返されます。

```
{
  "messages": [
    "警告メッセージ"
  ]
}
```

13.4. Webデータベース：レコードの削除（DELETE）

1件のレコードを削除できます。

13.4.1. HTTPメソッド

DELETE

13.4.2. URI

- パラメータを HTTP のクエリ文字列で送信する場合

http(s)://(POWER EGGサーバ)/pe4j/api/rest/v1/xdb/q/record.json

- パラメータを JSON形式で送信する場合

http(s)://(POWER EGGサーバ) /pe4j/api/rest/v1/xdb/record.json

13.4.3. リクエストパラメータ

パラメータ名	指定する値	必須	説明
system	数値	省略可	システム区分を指定します。 (0:WebDB,2:CRM) ※省略時は0が指定されます。
database	文字列	必須	データベース名を指定します。((親フォルダ名)/(フォルダ名)/(データベース名))
notification	真偽値	省略可	通知の有無を指定します。(true:通知する,false:通知しない) ※省略時はfalseが指定されます。
external	真偽値	省略可	外部連携（アドオン呼び出し）の有無を指定します。(true:実行する,false:実行しない) ※省略時はtrueが指定されます。
No	数値	必須	レコードNoを指定します。

13.4.4. リクエストの例

1. レコード更新条件

- 対象のデータベースは「ポイント管理型/クレーム管理」
- 通知の有無は「通知しない」
- 外部連携の有無は「実行する」
- 削除するレコードNoは「1」

2. パラメータを HTTP のクエリ文字列で送信する場合

HTTPのクエリ文字列の例

```
system=0&database=%EF%BE%8C%EF%BE%9F%EF%BE%9B%EF%BD%BE%EF%BD%BD%E7%AE%A1%E7%90%86%E5%9E%8B%2F%
```

リクエストヘッダを含んだ例

```
DELETE /pe4j/api/rest/v1/com/q/record.json?(HTTPのクエリ文字列) HTTP/1.1
Host: pe280demo.poweregg.net
X-API-Key: 7706d1ce-01a0-463b-a162-d389cec21eb1
```

❗ 注釈

Content-Typeヘッダは不要です。

3. パラメータを JSON形式で送信する場合

リクエストヘッダの例

```
DELETE /pe4j/api/rest/v1/com/record.json HTTP/1.1
Host: pe280demo.poweregg.net
X-API-Key: 7706d1ce-01a0-463b-a162-d389cec21eb1
Content-Type: application/json
```

❗ 注釈

Content-Type に application/json を指定して下さい。指定しない場合は JSON が解釈できないため、実行時エラーとなります。

リクエストボディの例

```
{
  "system": 0,
  "database": "フーズ管理型/クレーム管理DB",
  "notification": false,
  "external": true,
  "No": 1
}
```

13.4.5. レスポンスの例

処理成功時は空の JSON データが返されます。

```
{}
```

❗ 注釈

外部連携を実行する場合に「レコードの登録、変更、削除時」の処理で警告が返された場合は警告メッセージが返されます。

```
{
  "messages": [
    "警告メッセージ"
  ]
}
```

13.5. Webデータベース：フィールド形式

フィールドは以下のようなJSON形式になります。

type と value の部分がフィールドの型によって異なります。

なお、登録、更新APIでJSONデータを送信する場合、「value」パラメータのみ送信してください。「type」パラメータは必要ありません。

13.5.1. システム標準項目

フィールド型	type	value
No	85	取得時のレスポンスのデータ <pre>"No": { "type": 85, "value": (No[数値]) }</pre>

フィールド型	type	value
登録日時	86	- 取得時のレスポンスのデータ <pre> "登録日時": { "type": 86, "value": "(入力値[YYYY-MM-DDTHH:MM:SS±HH:MM])" }</pre> - 登録・更新時のリクエストデータ <pre> "登録日時": { "value": "(入力値[YYYY-MM-DDTHH:MM:SS±HH:MM])" }</pre>
登録者	87	- 取得時のレスポンスのデータ <pre> "登録者": { "type": 87, "value": { "code": "(入力値[ユーザID])", "name": "(入力値[社員名])" } }</pre> - 登録・更新時のリクエストデータ <pre> "登録者": { "value": { "code": "(入力値[ユーザID])" } }</pre>

フィールド型	type	value
最終更新日時	88	- 取得時のレスポンスのデータ <pre> "最終更新日時": { "type": 88, "value": "(入力値[YYYY-MM-DDTHH:MM:SS±HH:MM])" }</pre> - 登録・更新時のリクエストデータ <pre> "最終更新日時": { "value": "(入力値[YYYY-MM-DDTHH:MM:SS±HH:MM])" }</pre>
最終更新者	89	- 取得時のレスポンスのデータ <pre> "最終更新者": { "type": 89, "value": { "code": "(入力値[ユーザID])", "name": "(入力値[社員名])" } }</pre> - 登録・更新時のリクエストデータ <pre> "最終更新者": { "value": { "code": "(入力値[ユーザID])" } }</pre>
申請受付番号	90	取得時のレスポンスのデータ <pre> "申請受付番号": { "type": 90, "value": (申請受付番号[数値]) }</pre>

フィールド型	type	value
申請者	91	取得時のレスポンスのデータ <pre> "申請者": { "type": 91, "value": { "code": "(入力値[社員ID])", "name": "(入力値[社員名])" } } </pre>
申請日	92	取得時のレスポンスのデータ <pre> "申請日": { "type": 92, "value": "(入力値[YYYY-MM-DD])" } </pre>
申請部門	93	取得時のレスポンスのデータ <pre> "申請部門": { "type": 93, "value": { "code": "(入力値[部門ID])", "name": "(入力値[部門名(略称)])" } } </pre>
決裁状況	94	取得時のレスポンスのデータ <pre> "決裁状況": { "type": 94, "value": "(入力値[状況区分])" } </pre>

フィールド型	type	value
決裁者	95	取得時のレスポンスのデータ <pre> "決裁者": { "type": 95, "value": { "code": "(入力値[社員ID])", "name": "(入力値[社員名])" } } </pre>
決裁日	96	取得時のレスポンスのデータ <pre> "決裁日": { "type": 96, "value": "(入力値[YYYY-MM-DD])" } </pre>
申請番号	97	取得時のレスポンスのデータ <pre> "申請番号": { "type": 97, "value": "(申請番号)" } </pre>
申請件名	100	取得時のレスポンスのデータ <pre> "申請件名": { "type": 100, "value": "(入力値)" } </pre>

フィールド型	type	value
顧客リレーション	53	- 取得時のレスポンスのデータ <pre> "顧客リレーション": { "type": 53, "value": { "enterpriseID": (入力値[企業ID]), "postID": (入力値[部署ID]), "personID": (入力値[担当者ID]), "enterpriseName": "(入力値[企業名])", "postName": "(入力値[部署名])", "personName": "(入力値[担当者名])" } } </pre> - 登録・更新時のリクエストデータ <pre> "顧客リレーション": { "value": { "enterpriseID": (入力値[企業ID]), "postID": (入力値[部署ID]), "personID": (入力値[担当者ID]) } } </pre>

13.5.2. ユーザ定義項目

フィールド型	type	value
文字列（1行）	1	- 取得時のレスポンスのデータ <pre> "(フィールド名)": { "type": 1, "value": "(入力値[文字列])" } </pre> - 登録・更新時のリクエストデータ <pre> "(フィールド名)": { "value": "(入力値[文字列])" } </pre>

フィールド型	type	value
数値	2	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 2, "value": (入力値[数値]) } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": (入力値[数値]) } </pre>
日付	5	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 5, "value": "(入力値[YYYY-MM-DD])" } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": "(入力値[YYYY-MM-DD])" } </pre>
年月	6	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 6, "value": "(入力値[YYYY-MM])" } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": "(入力値[YYYY-MM])" } </pre>

フィールド型	type	value
時刻	9	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 9, "value": "(入力値[HH:MM])" } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": "(入力値[HH:MM])" } </pre>
文字列（複数行）	11	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 11, "value": "(入力値[文字列])" } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": "(入力値[文字列])" } </pre>
自動計算	12	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 12, "value": (入力値[数値]) } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": (入力値[数値]) } </pre>

フィールド型	type	value
自動採番	13	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 13, "value": (入力値[数値]) } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": (入力値[数値]) } </pre> 注意: 登録時に自動採番項目のフィールドを指定しない場合、自動採番項目に値は設定されません。また、登録、更新時にはvalueで指定した値ではなく、システム側で採番された値で更新されます。
リッチテキスト	14	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 14, "value": "(入力値[文字列])" } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": "(入力値[文字列])" } </pre>

フィールド型	type	value
フラグ	20	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 20, "value": (入力値[true/false]) } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": (入力値[true/false]) } </pre>
チェックボックス	21	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 21, "value": ["(入力値1[文字列])", ... "(入力値n[文字列])"] } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": ["(入力値1[文字列])", ... "(入力値n[文字列])"] } </pre>

フィールド型	type	value
ラジオボタン	22	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 22, "value": "(入力値[文字列])" } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": "(入力値[文字列])" } </pre>
ドロップダウン	23	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 23, "value": "(入力値[文字列])" } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": "(入力値[文字列])" } </pre>

フィールド型	type	value
D B 参照	24	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 24, "value": { "code": "(入力値[コード])", "name": "(入力値[名称])" } } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": { "code": "(入力値[コード])" } } </pre>
D B 参照コード入力	25	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 25, "value": { "code": "(入力値[コード])", "name": "(入力値[名称])" } } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": { "code": "(入力値[コード])" } } </pre>

フィールド型	type	value
WebDB参照	26	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 26, "value": { "code": "(入力値[コード])", "name": "(入力値[名称])" } } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": { "code": "(入力値[コード])" } } </pre>
WebDB参照コード入力	27	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 27, "value": { "code": "(入力値[コード])", "name": "(入力値[名称])" } } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": { "code": "(入力値[コード])" } } </pre>

フィールド型	type	value
会社選択	31	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 31, "value": { "corpCode": "(入力値[外部会社コード])", "name": "(入力値[会社名(略称)])" } } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": { "corpCode": "(入力値[外部会社コード])" } } </pre>
部門選択	51	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 51, "value": { "corpCode": "(入力値[外部会社コード])", "deptCode": "(入力値[外部部門コード1])", "name": "(入力値[部門名(略称)])" } } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": { "corpCode": "(入力値[外部会社コード])", "deptCode": "(入力値[外部部門コード1])" } } </pre>

フィールド型	type	value
社員選択	52	- 取得時のレスポンスのデータ <pre> "(フィールド名)": { "type": 52, "value": { "corpCode": "(入力値[外部会社コード])", "deptCode": "(入力値[外部部門コード1])", "empCode": "(入力値[外部社員コード1])", "name": "(入力値[社員名])" } } </pre> - 登録・更新時のリクエストデータ <pre> "(フィールド名)": { "value": { "corpCode": "(入力値[外部会社コード])", "deptCode": "(入力値[外部部門コード1])", "empCode": "(入力値[外部社員コード1])" } } </pre>
お客様選択	53	- 取得時のレスポンスのデータ <pre> "(フィールド名)": { "type": 53, "value": { "enterpriseID": (入力値[企業ID]), "postID": (入力値[部署ID]), "personID": (入力値[担当者ID]), "enterpriseName": "(入力値[企業名])", "postName": "(入力値[部署名])", "personName": "(入力値[担当者名])" } } </pre> - 登録・更新時のリクエストデータ <pre> "(フィールド名)": { "value": { "enterpriseID": (入力値[企業ID]), "postID": (入力値[部署ID]), "personID": (入力値[担当者ID]) } } </pre>

フィールド型	type	value
商品選択	54	- 取得時のレスポンスのデータ <pre>"(フィールド名)": { "type": 54, "value": { "corpCode": "(入力値[外部会社コード])", "commodityCode": "(入力値[商品コード])", "name": "(入力値[商品名])" } }</pre> - 登録・更新時のリクエストデータ <pre>"(フィールド名)": { "value": { "corpCode": "(入力値[外部会社コード])", "commodityCode": "(入力値[商品コード])" } }</pre>

フィールド型	type	value
添付ファイル	55	- 取得時のレスポンスのデータ <pre>"(フィールド名)": { "type": 55, "value": [{ "fileKey": "(入力値1[fileKey])", "name": "(入力値1[ファイル名])", "size": (入力値1[ファイルサイズ]) }, ... { "fileKey": "(入力値n[fileKey])", "name": "(入力値n[ファイル名])", "size": (入力値n[ファイルサイズ]) }] }</pre> - 登録・更新時のリクエストデータ <pre>"(フィールド名)": { "value": [{ "fileKey": "(入力値1[fileKey])" }, ... { "fileKey": "(入力値n[fileKey])" }] }</pre>

フィールド型	type	value
イメージ	56	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 56, "value": { "fileKey": "(入力値[fileKey])", "name": "(入力値[ファイル名])", "size": (入力値[ファイルサイズ]) } } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": { "fileKey": "(入力値[fileKey])" } } </pre>
URL	57	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 57, "value": { "url": "(入力値[URL])", "name": "(入力値[URL文字列])" } } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": { "url": "(入力値[URL])", "name": "(入力値[URL文字列])" } } </pre>

フィールド型	type	value
日付時刻	58	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 58, "value": "(入力値[YYYY-MM-DDTHH:MM±HH:MM])" } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": "(入力値[YYYY-MM-DDTHH:MM±HH:MM])" } </pre>
時間量	59	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 59, "value": (入力値[数値(分換算)]) } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": (入力値[数値(分換算)]) } </pre>
時間量計算	60	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 60, "value": (入力値[数値(分換算)]) } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": (入力値[数値(分換算)]) } </pre>

フィールド型	type	value
日付時刻計算	61	- 取得時のレスポンスのデータ <pre>"(フィールド名)": { "type": 61, "value": "(入力値[YYYY-MM-DDTHH:MM±HH:MM])" }</pre> - 登録・更新時のリクエストデータ <pre>"(フィールド名)": { "value": "(入力値[YYYY-MM-DDTHH:MM±HH:MM])" }</pre>

フィールド型	type	value
複数社員選択	62	• 取得時のレスポンスのデータ <pre>"(フィールド名)": { "type": 62, "value": [{ "corpCode": "(入力値1[外部会社コード])", "deptCode": "(入力値1[外部部門コード1])", "empCode": "(入力値1[外部社員コード1])", "name": "(入力値1[社員名])" }, ... { "corpCode": "(入力値n[外部会社コード])", "deptCode": "(入力値n[外部部門コード1])", "empCode": "(入力値n[外部社員コード1])", "name": "(入力値n[社員名])" }] }</pre> • 登録・更新時のリクエストデータ <pre>"(フィールド名)": { "value": [{ "corpCode": "(入力値1[外部会社コード])", "deptCode": "(入力値1[外部部門コード1])", "empCode": "(入力値1[外部社員コード1])" }, ... { "corpCode": "(入力値n[外部会社コード])", "deptCode": "(入力値n[外部部門コード1])", "empCode": "(入力値n[外部社員コード1])" }] }</pre>

フィールド型	type	value
スタンプ	63	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 63, "value": { "result": "(入力値[スタンプ名])", "corpCode": "(入力値[外部会社コード])", "deptCode": "(入力値[外部部門コード1])", "empCode": "(入力値[外部社員コード1])", "name": "(入力値[社員名])", "date": "(入力値[スタンプ処理日[YYYY-MM-DDTHH:MM±HH:MM]])" } } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": { "result": "(入力値[スタンプ名])", "corpCode": "(入力値[外部会社コード])", "deptCode": "(入力値[外部部門コード1])", "empCode": "(入力値 外部社員コード1)", "date": "(入力値[スタンプ処理日[YYYY-MM-DDTHH:MM±HH:MM]])" } } </pre>
WebDB間連携	64	- 取得時のレスポンスのデータ <pre> (フィールド名): { "type": 64, "value": (入力値[コード]) } </pre> - 登録・更新時のリクエストデータ <pre> (フィールド名): { "value": (入力値[コード]) } </pre>

13.5.3. WebDBのフィールド毎の利用可能な検索条件演算子

フィールド形式	比較方法	比較条件 (演算子)
すべての型		未入力 (n)
		入力済 (!n)
文字列一行, DB参照, DB参照コード入力, WebDB参照, WebDB参照コード入力, DB間連携, URL, 申請番号, 申請件名	文字比較	と同じ (=)
		と異なる (!=)
		で始まる (^)
		を含む (~)
		を含まない (!~)
文字列複数行	文字比較	で始まる (^)
		を含む (~)
		を含まない (!~)
ラジオボタン, ドロップダウン, 会社選択, 商品選択, スタンプ, 決裁状況	文字比較	と同じ (=)
		と異なる (!=)
リッチテキスト, チェックボックス	文字比較	を含む (~)
		を含まない (!~)
数値, 自動計算, 自動採番, 時間量, 時間量計算, レコードNo, 申請受付番号	数値比較	と同じ (=)
		と異なる (!=)
		より大きい (>)
		より小さい (<)
		以上 (>=)
		以下 (<=)
日付, 年月, 時刻, 日付時刻, 日付時刻計算, 登録日時, 最終更新日時, 申請日, 決裁日	日付比較	と同じ (=)
		と異なる (!=)
		以降 (>=)
		以前 (<=)
フラグ	フラグ	チェック済み (true)
		未チェック (false)
社員選択, 登録者, 最終更新者, 申請者, 決裁者	社員比較	と同じ (=)
		と異なる (!=)

フィールド形式	比較方法	比較条件 (演算子)
部門選択、申請部門	部門比較	と同じ (=)
		と異なる (!=)
		配下 (^=)
複数社員選択	複数社員選択	を含む (~)
		を含まない (!~)
お客様選択、顧客リレーション	お客様選択	企業が同じ (c=)
		部署が同じ (b=)
		担当者が同じ (e=)

13.6. Webデータベース：フィールド定義の取得（GET）

指定されたシステム区分、データベース名のデータベースに登録されている フィールド定義を一覧で取得できます。

13.6.1. HTTPメソッド

GET

13.6.2. URI

http(s)://(POWER EGGサーバ)/pe4j/api/rest/v1/xdb/q/fields.json

13.6.3. リクエストパラメータ

パラメータ名	指定する値	必須	説明
system	数値	必須	システム区分を指定します。（0:WebDB,1:汎用申請,2:CRM）
database	文字列	必須	データベース名を指定します。（(親フォルダ名)/(フォルダ名)/(データベース名)）

13.6.4. リクエストの例

1. レコード取得条件

- 対象のデータベースは「ポイント管理型/クレーム管理」

2. パラメータを HTTP のクエリ文字列で送信する場合

HTTPのクエリ文字列の例

```
system=0&database=%EF%BE%8C%EF%BE%9F%EF%BE%9B%EF%BD%BE%EF%BD%BD%E7%AE%A1%E7%90%86%E5%9E%8B%2F%
```

<URLエンコーディングする前>

```
system=0&database=7° 0tλ管理型/クレーム管理DB
```

リクエストヘッダを含んだ例

```
GET /pe4j/api/rest/v1/xdb/q/fields.json?(HTTPのクエリ文字列) HTTP/1.1
Host: pe280demo.poweregg.net
X-API-Key: 7706d1ce-01a0-463b-a162-d389cec21eb1
```

※Content-Typeヘッダは不要です。

13.6.5. レスポンス(JSON)

パラメータ名	データ型	必須	説明
id	数値		フィールドID
fieldName	数値		フィールド名
fieldType	日付		フィールド型
itemClass	日付		項目種別 (0 : システム項目、1 : ユーザ定義項目)
required	日付		必須フラグ (0 : 必須ではない、1 : 必須)

13.6.6. レスポンスの例

レコードのリストが JSON 配列で返されます。

```

{
  "fields": [
    {
      "id": 7283,
      "fieldName": "品目番号",
      "fieldType": 1,
      "itemClass": 1,
      "required": 0
    },
    {
      "id": 7284,
      "fieldName": "番号",
      "fieldType": 2,
      "itemClass": 1,
      "required": 0
    },
    {
      "id": 7285,
      "fieldName": "必須項目",
      "fieldType": 1,
      "itemClass": 1,
      "required": 1
    }
  ]
}

```

13.7. Webデータベース：レコード件数の取得（GET）

指定されたデータベースIDのデータベースに登録されている 条件に合致するレコードの件数を取得できます。

13.7.1. HTTPメソッド

GET

13.7.2. URI

http(s)://(POWER EGGサーバ)/pe4j/api/rest/v1/xdb/q/recordcount.json

13.7.3. リクエストパラメータ

パラメータ名	指定する値	必須	説明
corpId	文字列	必須	レコード件数を取得する対象のデータベースが配置されている会社の外部会社コードを指定します。
empCode	文字列	必須	外部社員コード 1 を指定します。
p1	数値	必須	データベースIDを指定します。
p2	数値	必須	一覧表示画面定義IDを指定します。
p3	数値	必須	絞り込み条件IDを指定します。

パラメータ名	指定する値	必須	説明
p4	文字列	省略可	フィールド定義のID\$フィールドの値\$比較演算子コード(ConditionType 0～99)を指定します。比較演算子コードの詳細は「WebDBのフィールド毎の利用可能な比較演算子コード」を参照してください。複数の条件を指定する場合は で繋げます。すべてAND条件となります。(OR条件は指定不可)

❶ 注釈

データベースID、一覧表示画面定義ID、絞り込み条件IDは、レコード一覧画面の「この情報のURL」から確認が可能です。

例) /XDB/XDB0102l.jsf?p1=101134&p2=1334&p3=1174

- ・データベースID⇒p1=101134
- ・一覧表示画面定義ID⇒p2=1306
- ・絞り込み条件ID⇒p3=1174

13.7.4. WebDBのフィールド毎の利用可能な比較演算子コード

フィールド形式	比較方法	比較条件 (演算子)	比較演算子コード
すべての型 (チェックボックス、申請番号、申請件名を除く)		未入力 (n)	0
		入力済 (!n)	99
文字列一行、URL	文字比較	と同じ (=)	1
		と異なる (!=)	2
		で始まる (^)	3
		を含む (~)	4
		を含まない (!~)	5
文字列複数行	文字比較	で始まる (^)	3
		を含む (~)	4
		を含まない (!~)	5
ラジオボタン、ドロップダウン、DB参照、DB参照コード入力、WebDB参照、WebDB参照コード入力、決裁状況、WebDB間連携項目、スタンプ	文字比較	と同じ (=)	1
		と異なる (!=)	2
リッチテキスト、チェックボックス	文字比較	を含む (~)	4
		を含まない (!~)	5

フィールド形式	比較方法	比較条件 (演算子)	比較演算子コード
数値、自動計算、自動採番、時間量、時間量計算、レコードNo、申請受付番号	数値比較	と同じ (=)	1
		と異なる (!=)	2
		より大きい (>)	6
		より小さい (<)	7
		以上 (>=)	8
		以下 (<=)	9
日付、年月、時刻、日付時刻、日付時刻計算、登録日時、最終更新日時、申請日、決裁日	日付比較	と同じ (=)	10
		と異なる (!=)	11
		以降 (>=)	8
		以前 (<=)	9
フラグ	フラグ	チェック済み (true)	12
		未チェック (false)	13
申請番号、申請件名	文字比較	と同じ (=)	1

❗ 注釈

DB参照, DB参照コード入力, WebDB参照, WebDB参照コード入力, WebDB間連携項目の項目はコード値での比較となります。

13.7.5. リクエストの例

1. 件数取得条件

- 対象のデータベースはデータベースID=123456
- 一覧表示画面定義は一覧表示画面定義ID=1431
- 絞り込み条件は絞り込み条件ID=1001
- フィールド条件はフィールド定義ID=249（形式：数値）、フィールドの値=1000

2. パラメータを HTTP のクエリ文字列で送信する場合

HTTPのクエリ文字列の例

```
corpId=friend00&empCode=ooishi&p1=123456&p2=1431&p3=1001&p4=249%241000%241
```

<URLエンコーディングする前>

```
corpId=friend00&empCode=ooishi&p1=123456&p2=1431&p3=1001&p4=249$1000$1
```

リクエストヘッダを含んだ例

```
GET /pe4j/api/rest/v1/xdm/q/recordcount.json?(HTTPのクエリ文字列) HTTP/1.1
Host: pe280demo.poweregg.net
X-API-Key: 7706d1ce-01a0-463b-a162-d389cec21eb1
```

※Content-Typeヘッダは不要です。

13.7.6. レスポンス(JSON)

パラメータ名	データ型	必須	説明
recordCount	数値	必須	条件に合致するレコード件数

13.7.7. レスポンスの例

レコードの件数が JSON 形式で返されます。

```
{
  "recordCount": 8
}
```